

Hjemmeopgave

Byg en service, der beriger virksomhedsdata med branchestatistik. Servicen skal integrere data fra to eksterne kilder (CVR API og Danmarks Statistik), og eksponere et samlet, forenklet API. Du skal designe og implementere en service, som opfylder nedenstående user stories og acceptkriterier:

User stories

Som consumer af servicen vil jeg gerne kunne:

- slå en virksomhed op via virksomhedens CVR-nummer og/eller navn
- modtage virksomhedens stamdata for et givent år beriget med branchestatistik i ét response

Acceptkriterier:

- Virksomhedens stamdata udgøres af navn, adresse, branchekode og antal ansatte
- Branchestatistik udgøres af antal virksomheder, ansatte, samlet beskæftigelse og lønsum

Som driftansvarlig vil jeg gerne kunne opdage problemer og driftsforstyrrelser proaktivt ved at:

- overvåge servicens sundhed og performance gennem metrikker
- overvåge de eksterne datakilders sundhed og performance gennem metrikker

Acceptkriterier:

- Metrikker udgøres af responstid (95. percentil) og fejlrate

Eksterne datakilder

CVR API

Virksomhedsdata, inklusiv branchekode (industrycode) som bruges i det efterfølgende kald til Danmarks Statistik, hentes fra public API'er på <https://apicvr.dk>, eksempelvis:

- Hent virksomhed via CVR: <https://apicvr.dk/api/v1/{cvrNumber}>
- Hent virksomhed via navn: <https://apicvr.dk/api/v1/search/company/{companyName}>

Danmarks Statistik

Branchestatistik hentes fra public API'er på <https://api.statbank.dk>:

- Hent branchedata ud fra branchekode: <https://api.statbank.dk/v1/data> med POST body

```
{
  "table": "ERHV1",
  "format": "CSV",
  "variables": [
    {
      "code": "BRANCHE07",
      "values": [<branchekode>]
    },
    {
      "code": "TAL",
      "values": [
        "ARBSTED",
        "ANSATTE",
        "FULDBESK",
        "LØNSUM"
      ]
    },
    {
      "code": "TID",
      "values": [<år>]
    }
  ]
}
```

Tekniske krav

Sprog

- Vi bruger C# i vores stack, men du er velkommen til at vælge det sprog, du er mest komfortabel i
- API'er udstilles i REST og/eller GraphQL

Kvalitet

- Inkluder unit tests – fuld testdækning er ikke nødvendig, men vi vil gerne se noget, der demonstrerer din teststrategi
- Håndter fejl i downstream-systemer robust

Drift og deployment

- Applikationen skal kunne bygges og køres via Docker
- Inkluder en CI-pipeline (f.eks. ved hjælp af Github Actions), der bygger, tester og publicerer et container image

Aflevering

- Upload løsningen til et offentligt repository f.eks. GitHub
- Inkluder en README fil i dit repository, der indeholder instruktioner til at køre applikationen lokalt og afprøve servicen
- Beskriv også (kort) overvejelser om til- og fravalg i README-filen.

Scope og rammer

Udover ovenstående tekniske krav, har du frie hænder til at implementere løsningen, som du ønsker. Tredjepartspakker, libraries, LLM'er og lignende er tilladt. Der er selvfølgelig ikke tale om en produktionsklar løsning, så fokusér på det, du synes er vigtigst. Brug den tid, du finder nødvendig, men vi anbefaler dog ikke at bruge mere end en dag på opgaven.

Samtalen

Til den efterfølgende samtale vil vi bede dig præsentere løsningen og dine overvejelser, gerne oppe ved tavlen. Formålet er ikke at evaluere en produktionsklar løsning, men at have en god og konstruktiv dialog om dine prioriteringer og valg. Samtalen tager udgangspunkt i den implementerede løsning, og vi vil især lægge vægt på dine designovervejelser og tanker om yderligere tiltag i et mere produktionslignende setup. Der er afsat ca. 30 min til denne samtale.

Vi ser frem til at tale videre med dig!